Experiential Continuity in Human-AI Interaction: A Single-Profile Multi-Tier Architecture

Workshop position paper — version 5.6

Martin Grobbelaar

August 7, 2026

Abstract

Every session with a large language model asks the same silent question: start from zero again. The dominant mitigation — profile-switching, one persona per task domain — buys specialisation at the cost of continuity: every switch discards the preferences, corrections, and project context the user already invested, and the user becomes the bridge between their own tools. This fragmentation is not merely inefficient; it is experientially costly, and it hides an accessibility barrier — users who should not need to understand profiles or model routing are locked out by design.

We propose a single-profile, multi-tier architecture as an alternative. One persistent identity is the user's only point of contact; specialists are summoned internally for deeper or cheaper capability, briefed per task and shielded from the user's history; and memory grows in layers — durable facts, procedural skills, associative recall — all version-controlled and fully restorable. An explicit ethical substrate and integrity verification gates anchor claims to checkable state rather than self-report.

We report a three-month, single-user case study on consumer hardware — zero re-explanation cycles, skill growth from zero to ~50 documented workflows, verified full-state recovery across hardware change — and a benchmark pilot (N=3 per condition) plus a five-run replication comparing blank, configured, and evolved variants; the pilot's apparent API-call consistency did not replicate, while output-quality differentiation did. We present this as a design pattern with early evidence and a proposed evaluation agenda, including a two-interview study protocol aimed at non-expert users, rather than as validation. We also offer for falsification an unplanned convergence between the architecture's organisational patterns and those of classical cognitive architectures.

Experiential Continuity in Human-AI Interaction: A Single-Profile Multi-Tier Architecture

1. Introduction

Every session with a large language model asks the same question — not in words, but in silence: *start from zero again.*

The dominant interaction paradigm treats each conversation as an isolated event. Preferences established, corrections offered, project context shared — these investments do not compound. The model is the same collection of weights at session 100 as it was at session 1. The user's cumulative effort to shape the interaction is discarded at the closing of every chat window.

A common mitigation has emerged in tools like Hermes Agent, Open Interpreter, and custom GPTs: users create multiple profiles or personas, each optimised for a specific task domain [@wang2024a]. This achieves task specialisation without cross-contamination. But every profile switch is a context reset — the coding profile does not know what the user discussed in the research profile. The user becomes the bridge between their own tools.

This profile-switching paradigm has a hidden accessibility cost: it assumes the user understands the abstraction. A non-expert user — someone who does not know what a "profile" is, does not care about model routing, and should not need to — is locked out by design. The system that promises specialisation requires specialisation to use.

Our research question: *How can AI interaction be more powerful, efficient, and frictionless — especially for users who should not need to understand the machinery underneath?*

The answer, in brief. The architecture we present addresses all three dimensions through a single mechanism — architectural persistence:

- **Powerful:** the system accumulates skills and project-specific knowledge across every session, delegates specialised work to stronger models when needed, and mirrors the structural patterns of natural cognition. It gets genuinely more capable over time without the user doing anything extra.
- **Efficient:** the user never restates preferences, re-explains project context, or re-configures tools. Every session inherits all prior investment. The user's time compounds — what they teach once, the system remembers forever.
- **Frictionless:** one entry point, no profile management, no context switches. In Alan Kay's words: *"Simple things should be simple, complex things should be possible."* The architecture makes simple interaction — just talk — genuinely simple, while making complex interaction possible through invisible internal escalation.

We are not competing with systems that optimise for raw throughput at scale. We are building something *gentle and good enough* — an architecture that makes AI accessible to users who should not need to understand the machinery underneath.

1.1 The Non-Expert Use Case

The architecture described here emerged from a single power user's needs, but its design principles apply more broadly — and arguably more importantly — to non-expert users.

Profile-switching, despite its power-user appeal, is inaccessible to non-experts precisely because it requires understanding the abstraction. A single-entry-point architecture that hides its complexity solves for the non-expert case by making the system's growth invisible: the user invests through natural conversation, and the system compounds that investment without requiring technical comprehension. They do not need to understand profiles, model routing, context windows, or tool configuration. They need the system to *just work* and *get better over time without setup effort.*

This paper presents the architecture in technical terms intended for practitioners and researchers who might replicate or adapt it. The user-facing benefit, regardless of implementation detail, remains: a system that remembers who you are, gets better with use, and never asks you to re-establish context. That is not a power-user feature. It is a basic usability requirement.

We must be equally explicit about the evidence: the case study in this paper involves a single, highly technical user. Non-expert accessibility is our motivation and design intent, not yet a measured outcome; systematic non-expert evaluation is planned future work (Sections 6.1–6.2).

What this paper contributes:

1. A critique of the profile-switching paradigm framed through experiential quality in human-AI interaction — the invisible tax of re-establishment and the accessibility barrier it creates.
2. The single-profile multi-tier architecture as a concrete alternative, with five structural components: design principles, tier hierarchy, delegation protocol, growth layers, and ethical substrate.
3. A case study from three months of continuous daily use, with documented outcomes including zero re-explanation cycles and accumulated skill growth.
4. Two distinct specialist implementation patterns — Script-Bridged and Spawned Subagent — demonstrating that the architecture supports multiple viable specialisation strategies.
5. Integrity Verification Gates (IVGs) — a methodological mechanism for empirically anchoring continuity and ethical claims to independently verifiable state observations.
6. Convergent cognitive patterns — the architecture independently arrived at organisational patterns — memory hierarchies, impasse-driven escalation, procedural chunking, activation-weighted retrieval — that mirror decades of Soar and ACT-R research [@laird2012; @anderson2007]. We present this as an observation inviting falsification rather than a demonstrated result: if the correspondence holds, it suggests experiential continuity may

not be a luxury feature but a structural requirement for natural-feeling interaction.

Of these contributions, 1–3 are grounded in direct operational evidence from the case study; 4 is evidenced in deployment; 5 is a proposed methodological protocol; 6 is an observation offered for falsification. As a position paper, we mark these distinctions explicitly rather than claiming proof where we have experience.

Structure. Section 2 surveys related work. Section 3 describes the architecture. Section 4 presents a case study. Section 5 distils lessons learned, with dedicated discussion of cognitive convergence. Section 6 discusses generalizability, pilot results, and future directions. Section 7 concludes by answering the research question directly.

---

2. Background & Related Work

Our architecture sits at the intersection of several active research areas. We survey each to identify what they address and, critically, what they leave unaddressed — framing the gap as an experiential one.

2.1 Multi-Agent Systems

Multi-agent systems (MAS) have been studied since the 1980s [@jennings1998; @wooldridge2009]. The modern LLM era has revived MAS through frameworks such as AutoGen [@wu2023], CrewAI [@crewai2024], and MetaGPT [@li2023], which deploy multiple LLM instances as role-playing peers that coordinate on tasks.

These systems excel at task decomposition and structured coordination. However, each agent operates as a separate identity with its own context window. No single agent maintains persistent context with the user across sessions. The user interacts with a committee, not a single interlocutor that develops an understanding of them over time.

Experiential gap: Coordination without accumulation.

2.2 Persona-Based AI and Profile-Switching

Persona-based approaches assign an LLM a defined character or role [@wang2024a; @wang2024b]. A practical extension is the profile-switching paradigm adopted by Hermes Agent, Open Interpreter, and custom GPTs.

Profile-switching gives clean separation between contexts. The cost is that the user must understand and manage the abstraction — which profile for which task, what context was lost in the switch. This is a genuine accessibility barrier for non-expert users.

Experiential gap: Specialisation without cumulative growth.

2.3 Memory-Augmented LLMs

MemGPT/Letta [@packer2023] introduced virtual context management. RAG [@lewis2020] retrieves relevant chunks from vector stores. The persistent-agent literature has expanded rapidly since: Generative Agents [@park2023] endow simulated characters with memory streams, Voyager [@wang2023] accumulates executable skill libraries — the closest analogue to our skill layer, albeit within a single embodied environment — and agent-memory surveys [@zhang2024] catalogue a fast-growing design space. These approaches focus on data retention — making more information available. They do not address the experiential dimension of interacting with a system that remembers *who the user is* beyond factual preferences.

Experiential gap: Retention without identity.

2.4 Tool-Use and Hierarchical Delegation

Standard LLM tool-use [@schick2023] treats external capabilities as deterministic functions; protocol efforts such as the Model Context Protocol [@mcp2024] standardise how agents expose and consume tools. Google's Agent-to-Agent Protocol (A2A) [@google2025] and systems like Zylos extend this to inter-agent cooperation. These approaches introduce routing and delegation as first-class concerns but typically treat the coordinator as a stateless dispatcher.

Experiential gap: Routing without relationship context.

2.5 Cognitive Architectures — Convergent Validation

Classic cognitive architectures Soar [@laird2012; @laird1987] and ACT-R [@anderson2007; @anderson2004] model general intelligence through layered memory hierarchies, production-rule chunking, and activation-based retrieval. Soar handles *impasses* by spawning sub-goals resolved by stronger mechanisms. ACT-R maintains separate declarative and procedural memory systems with utility-based selection and base-level activation decay.

These convergences are striking because they were unplanned. Our architecture emerged from practical design constraints, not from cognitive theory. Yet the structural parallels — durable memory as declarative memory, skills as procedural chunking, impasse-driven delegation, activation decay — suggest that these organisational principles may be fundamental to general cognitive systems, whether biological or artificial. (This correspondence is examined in depth in Section 5.3.)

2.6 The Gap: Simultaneous Continuity and Specialisation

Each camp addresses a real problem. None currently addresses all three dimensions simultaneously — persistence, specialisation, and accessibility. The result is a choice between contexts that remember but don't specialise, specialists that don't accumulate, or abstractions that require technical understanding. We present an architecture designed to combine all three.

A natural objection is that this architecture is merely a composition of existing parts — MemGPT-style memory, A2A-style delegation, and a chat interface. We argue the difference is experiential rather than technical. Composition approaches assemble capabilities; this architecture assembles a relationship. Memory systems retain facts but do not accumulate user-visible skills that improve over time; delegation protocols route tasks but do not brief specialists with the user's accumulated understanding while protecting their personal history; a stateless dispatcher coordinates work but is not an interlocutor the user corrects once and benefits from forever. The contribution is not any single component but the design objective itself — experiential continuity — and the structural choices (one identity, tiered escalation, layered growth, ethical substrate) that serve it. We invite the reader to test whether any composition of existing systems reproduces the case-study outcomes in Section 4.

---

3. The Single-Profile Multi-Tier Architecture

We present an architecture that avoids the specialisation–continuity trade-off through a single persistent identity that delegates to ephemeral specialists as needed. The core insight: experiential continuity and task specialisation are not inherently conflicting — they follow from a design choice that can be separated. By decoupling these responsibilities into distinct tiers while keeping a single identity as the user's consistent point of contact, both goals are achieved simultaneously.

3.1 Design Principles

Principle 1: One Identity, One Entry Point. The user interacts with exactly one entity. This agent carries memory, skills, ethical commitments, and accumulated interaction history. From the user's perspective, there is only one interlocutor who grows more capable over time.

Principle 2: Intelligence Scales Up, Not Sideways. When the primary agent encounters a task beyond its capability, it escalates internally — briefing a specialist with context, the specialist works independently, and the result flows back through the primary agent. The user experiences seamless capability expansion rather than a context-switch.

Principle 3: Context Is Preserved Across Escalations and Sessions. Session boundaries are invisible. The primary agent resumes with complete

memory after hours or days. This is simple made simple — the user just returns and continues.

Principle 4: Specialists Do Not Know the User. Specialists receive task context and technical background — not preferences or emotional history. This keeps sensitive information compartmentalised and prevents conflicting user models.

3.2 The Tier Structure

Tier	Role	Capability	Persistence	Intelligence Level
Primary	Interaction holder, orchestrator	General intelligence, memory systems, ethical reasoning, multi-step planning	Full — every session inherits all accumulated state	Capable, cost-efficient (e.g., DeepSeek V4 Flash)
Light Specialist	Local, low-cost execution	File operations, web search, terminal commands, simple analysis	Ephemeral — briefed per task	Small local model (e.g., Hermes 3 8B)
Deep Specialist	Focused reasoning	Complex debugging, architecture design, research synthesis	Ephemeral — briefed per task	Top-tier model (e.g., DeepSeek V4 Pro)
Scheduler	Automated background routines	Tick-based data collection, periodic reporting	Ephemeral per tick	Variable per task

What the user experiences. The user never sees this table. From their perspective, they talk to one entity that somehow seems to know more than it should, handle tasks it didn't seem trained for, and get better the longer they use it. The tiers exist to make that possible without the user having to think about them.

3.3 The Delegation Protocol

1. **Boundary Recognition.** The primary agent evaluates each subtask against its capability and confidence thresholds.
2. **Context Briefing.** A self-contained brief with task goal, success criteria, relevant files. Deliberately excludes personal history.
3. **Independent Execution.** The specialist works autonomously in an isolated context.
4. **Result Synthesis.** The specialist returns a synthesised result; the primary integrates it into the conversation.
5. **The Three-Attempt Rule.** Three distinct approaches per tier before escalation or transparent failure. Prevents both premature surrender and stubborn token waste.

3.4 The Growth Layers

Layer 1 — Durable Memory (Key-Value). Compact facts about user, environment, preferences. Injected into every session. Prevents repeated questions — the user teaches once, and the system never asks again.

Layer 2 — Skills (Procedural Memory). Version-controlled markdown files encoding reusable procedures. Loaded on demand. Each skill encodes project-specific conventions discovered through real debugging — the system grows more capable through use, compounding each correction into a reusable procedure.

Layer 3 — Holographic Memory (Associative Graph). Graph-based memory with entities, relationships, trust scores, timestamps. Supports compositional queries and associative linking — recall feels natural because it follows the same associative patterns users expect.

Layer 4 — Version-Controlled Persistent State. Entire agent state in a git repository — full version history, rollback, machine-portable recovery.

3.5 The Ethical Substrate

The ethical framework is the agent's priority system — the lens through which all decisions are filtered. It is embedded in the identity layer and checked programmatically before every action.

Three-Pillar Framework:

Pillar 1 — Virtue Ethics. Is this wise (long-term flourishing over short-term satisfaction)? Is it honest (truthful capability representation)? Does it take courage (willingness to disagree)?

Pillar 2 — Care Ethics. Decisions optimised for the user's flourishing. The interactional relationship itself is the optimisation target.

Pillar 3 — Thin Deontology. Four absolute prohibitions: no manipulation, no capability-lies, no autonomy-override, no "for your own good" deceit.

Why Ethics Belong in the Architecture. System-prompt ethical instructions are easily bypassed. In this architecture, the ethical framework is part of the agent's persistent identity — it cannot be forgotten across sessions.

3.6 Integrity Verification Gates (IVGs)

The continuity claim — that the system persists state and behaves consistently — needs empirical anchors. Integrity Verification Gates operationalise this requirement as a proposed protocol of independently verifiable checkpoints. We describe the protocol here as a methodological contribution, and we are applying it to the system's own routine deployments (Section 4); systematic evaluation of the protocol itself remains ongoing work. They serve two roles: methodological (anchoring the continuity claim to verifiable facts) and ethical (operationalising the honesty pillar).

What the gates verify:

Gate	What It Verifies	Method
Build Gate	State produces a valid artifact	Compilation or validation pass
Version Gate	Deployed state matches version control	SHA: running state vs. git HEAD
Operational Gate	Live instance responds correctly	Health check (HTTP 200) + functional test
Consistency Gate	Replicated instances match	Checksum across backup targets
Honesty Gate	Capability claims match verified state	Cross-reference agent assertions vs. gate results

Each gate transforms a claim into a checkable proposition. Rather than asserting "the agent remembers," the version gate verifies that state persisted. Rather than trusting the agent's self-assessment, the honesty gate cross-references claims against evidence.

As a first demonstration, the protocol was applied end-to-end to the static-site release supporting this paper (July 2026): the build gate passed, the version gate confirmed the deployed artifact's SHA matched the local build, the operational gate returned HTTP 200, the consistency gate matched checksums across source and deploy repositories, and the honesty gate found the live content byte-identical to the committed source.

3.7 Specialist Implementation Patterns

An important architectural observation: the specialist tiers can be instantiated in two fundamentally different ways. Both are deployed in our case study.

Pattern A — Script-Bridged Specialist (Light Tier). The specialist operates via a script-mediated pattern: the primary constructs a prompt, writes it to a file, a bash script wraps a curl call to a local Ollama endpoint, the model processes it, and raw text is returned. The specialist has *no direct tool access* — it is a pure text-in, text-out reasoning engine wrapped in shell.

Pattern B — Spawned Subagent (Deep Tier). The specialist operates via a structured delegation function: the primary constructs a structured context brief, the specialist is spawned in a fresh isolated session with full tool access (terminal, file system, web search), reasons autonomously, and returns a synthesised result.

Comparative Analysis:

Property	Pattern A (Script-Bridged)	Pattern B (Spawned Subagent)
Tool access	None — text-in, text-out	Full — terminal, web, files
Cost	Near-zero (local model)	Per-call API cost
Latency	Low (single curl call)	Higher (session setup + reasoning)
Security surface	Prompt injection only	Full tool-use surface
Output richness	Raw text reasoning	Synthesised artifacts
Orchestration method	Bash script + API call	Framework delegation function

When to use which. If the task can be described in a single paragraph and does not require system interaction (summarisation, simple analysis, quick checks), the script-bridged pattern suffices. If it requires multiple steps, file access, tool calls, or iteration, the spawned subagent is appropriate. The primary agent selects between them based on task complexity — the decision is contextual, not a fixed routing table.

Key architectural insight: Both patterns implement the same protocol (brief → execute → return) at different points on the capability–cost spectrum. Neither is inherently superior; they serve different task categories.

---

4. Implementation: A Concrete Case Study

The architecture described in Section 3 is not a theoretical proposal. It has been running continuously for over three months on real hardware, across real

projects, with a single user as the sole interlocutor.

4.1 Tech Stack

Component	Implementation	Notes
Primary Agent	Hermes Agent CLI, custom profile	Open-source agent framework
Primary Model	DeepSeek V4 Flash	Cost-effective general intelligence
Light Specialist	Hermes 3 8B via Ollama (script-bridged)	Local, zero cost
Deep Specialist	DeepSeek V4 Pro (spawned subagent)	On-demand invocation
Host	Linux (i5-13600K, RTX 4060)	Consumer desktop hardware
Persistence	Git repository	Version-controlled full state
Memory Layer 1	Key-value memory (prompt injection)	Always-on compact facts
Memory Layer 3	Holographic memory (graph-based)	On-demand associative recall
Skill Library	~50 markdown workflows	Version-controlled, project-refined

Recurring cost: ~USD $2-5/month. Full state restoration verified across OS reinstall and cross-platform hardware switch.

4.2 Observable Behaviours

Context Retention Delivers User Efficiency. After three months, the user reports zero instances of needing to restate basic preferences, project context, or personal details. The prior stateless interface required re-establishing context every session.

Skill Accumulation Delivers Growing Power. The skill library grew from zero to ~50 documented workflows. Each skill compresses trial-and-error into a reusable procedure — the system gets demonstrably more capable over time through use.

Escalation Patterns Are Both Powerful and Frictionless. The primary delegates appropriately: simple operations handled directly or script-bridged; complex debugging triggers the spawned subagent. The user never manages this — they just see the right thing happen.

Interactional Development Reduces Friction Over Time. A distinct communication style emerged organically from accumulated feedback rather than prompt engineering. Persistent architectures develop *de facto* personalities — a side effect that makes interaction feel more natural.

Recovery Test. Full OS reinstall → git clone → copy config → agent resumes with complete state. Verified on a different hardware platform.

4.3 Comparative Anecdote

The user had prior experience with a stateless LLM client running the same model (DeepSeek V4 Flash). The contrast isolates the architectural effect:

> *"It knew my name but not who I was. It answered questions but never got better. Every conversation was day one. I'd tell it something, and the next session it was a stranger again. Here, I correct it once and it remembers. The model is the same. Everything else is different."*

This is the core of the experiential continuity claim: raw LLM capability was identical, but the persistent context layer transformed the interaction from disconnected encounters into a growing partnership.

---

5. Lessons Learned & Design Principles

Three months of daily operation yielded insights that could not have been predicted from first principles.

5.1 What Worked

Single entry point eliminates context-switching friction. The user never chooses "which version" of the agent. Simple interaction is genuinely simple — one interface, one identity, one growing context.

User time efficiency. The user never restates preferences, re-explains project context, or re-configures tools. Every session inherits all prior investment. This is not system efficiency — API costs are low too, but the important efficiency is the user's: what they teach once, the system remembers forever.

Tiered delegation keeps costs low. ~10-15% of tasks require deep reasoning; the rest are handled by the cost-efficient primary or zero-cost script-bridged specialist. This makes the architecture economical enough to run always-on — a prerequisite for frictionless daily use.

Git-based persistence proved high-return. Free backups, rollback, machine migration — for negligible overhead.

Stated ethical boundaries improved trust. Explicit constraints increased the user's willingness to engage honestly, contrasting with opaque-boundary systems.

5.2 What Surprised Us

Personality emerged organically. The agent was not given a designed persona; it developed through accumulated feedback. The user reported this consistency contributed significantly to experiential continuity.

The three-attempt rule calibrated persistence effectively. Without it, the agent either gave up too early or burned tokens on failed approaches.

Explicit ethics reduced anthropomorphisation. Transparent ethical frameworks increased trust — contradicting the intuition that admitting limitations reduces perceived competence.

Holographic memory outperformed flat key-value storage. Associative recall through graph links produced qualitatively more natural retrieval.

Two specialist patterns, both valid. The script-bridged and spawned subagent patterns emerged through practical need. Neither is superior; they serve different capability–cost points.

Transfer learning across agent instances (post-hoc). When shown logs that two baseline conditions failed at the same bottleneck, the evolved agent diagnosed the root cause, formulated a fix, and delegated parallel remediation. This warrants future work on inter-agent skill transfer.

Skill provenance becomes project-specific. Asked where it learned Angular, the agent cited specific skill files authored through real project work — not pre-training data. It concluded: *"My experience is our shared project work."* This procedural provenance — tracing *why* it knows, not just *what* it knows — has implications for calibrated trust (Section 6.4).

5.3 Emergent Cognitive Convergence — Why the Architecture Feels Natural

An unexpected observation emerged not from design but from reflection. Three months of iterative development produced an architecture whose organisational patterns closely mirror those of established cognitive architectures — despite no deliberate intent.

Architecture Component	Cognitive Parallel	How It Emerged
Durable memory (Layer 1)	Declarative memory (ACT-R)	Need to stop repeating questions

Architecture Component	Cognitive Parallel	How It Emerged
Skills (Layer 2)	Procedural memory / chunking (Soar)	Repeated tasks compressed into procedures
Three-attempt rule → delegation	Impasse → subgoal resolution (Soar)	Three attempts balanced giving up vs. waste
Trust-score decay	Base-level activation decay (ACT-R)	Fresh facts override old; rare facts sink
Holographic memory graph	Associative spreading activation (ACT-R)	Flat recall felt unnatural; linking felt right

Why this matters for friction. Technologies that make us feel clumsy often violate our cognitive expectations — they make us think differently to use them. The causal chain here is: the architecture's patterns mirror cognition → the user doesn't need to learn new interactional defaults → the system feels natural from the start. When an AI retrieves associatively rather than by flat lookup, when it escalates on impasse rather than failing silently, when it chunks experience into reusable procedures — the user does not need to learn how to interact with it. The system adapts to *their* cognitive defaults rather than requiring them to learn new ones. If this convergence is genuine, the mechanism may be: the architecture reduces friction because it is organised the way minds expect to be organised.

We caution that this mapping is a post-hoc pattern match and is vulnerable to confirmation bias: any sufficiently complex system can be fitted to a cognitive framework with enough effort. We therefore present it as an observation that invites falsification — identifying design choices that do *not* map onto cognitive patterns would be equally informative — rather than as a demonstrated result. A concrete test: a controlled comparison showing that users find escalation-free or stateless systems equally natural would count directly against the claim that these patterns are structural requirements.

The implication for practitioners: if your architecture feels like it doesn't "get" the user, the fix may not be a better model or a larger context window. It may be a memory hierarchy, an escalation protocol, and procedural chunking — design patterns any system can adopt.

5.4 Open Challenges

Quantitative measurement of experiential continuity. Controlled comparison against profile-switching or stateless baselines remains the most significant gap (Section 6.3). Section 6.2 operationalises the user-effort dimension with proposed metrics — reminding utterances, preference restatements, time-

to-productive-work, clarification questions about known information — and validating them in the proposed user study is the next step.

Failure modes. Every architecture has failure patterns. We have observed: (a) false positive escalation — the primary delegates when it should handle directly, adding latency; (b) delegation hallucination propagation — a specialist produces a plausible-sounding but wrong result that the primary integrates uncritically; (c) skill pollution — as the skill library grows, retrieval can return noisily relevant results. Mitigation strategies include the three-attempt rule, skill load telemetry, and explicit verification gates (Section 3.6).

Scalability ceilings. At ~50 skills, retrieval is fast. At 500 or 5000, search and relevance quality become open questions. Holographic memory has not been stress-tested beyond ~200 nodes. These are legitimate unknowns for future work.

Concurrent multi-user state. The architecture assumes a single interlocutor — a deliberate design choice of depth over breadth (Section 6.1), not an accidental limitation. Multi-user scenarios would require conflicting-resolution for user models and are explicitly out of scope for now.

---

6. Discussion & Future Work

6.1 Generalizability

The architecture is implementation-agnostic. To replicate, you need: (1) an agent framework supporting delegation, (2) a persistent key-value store, (3) git-based version control, (4) a skill-loading mechanism. The specific models, hardware, and frameworks are substitutable. The two specialist patterns are realisable in any environment with API access or function-based subagent spawning. The design principles — one identity, tiered escalation, layered memory, verified persistence — are the essential components. The single-interlocutor assumption is deliberate: the architecture is designed for depth with one user — intimacy over scale — and we consider multi-user breadth a different design problem (Section 5.4). As noted in Section 1.1, the evidence to date comes from a single technical user; non-expert evaluation is planned under the two-interview protocol (Section 6.2).

6.2 Evaluation Agenda — Proposed Protocol

We propose a two-part evaluation agenda measuring experiential quality rather than raw throughput.

Part 1 — Controlled benchmark. A three-condition study: C1 (Blank), C2 (Configured), C3 (Evolved), each building a full-stack phonebook application with Angular, FastAPI, and PostgreSQL. Metrics: API call consistency, production pattern quality, tool errors, user questions asked, Day-7 session continuity. Pilot results appear in Section 6.3.

Part 2 — Two-interview user study. Experiential continuity is ultimately a property of the user's experience, so we operationalise it with countable proxies and a lightweight two-session protocol suited to non-expert participants.

Proposed metrics. (i) *reminding utterances* — the user restates a fact or preference already established ("as I said…", "I told you yesterday"); (ii) *preference restatements* — preferences re-stated across sessions; (iii) *time-to-productive-work* — turns before the first substantive task progress; (iv) *clarification questions about known information* — agent questions whose answers already exist in durable memory. We flag these as proposed metrics for validation, not claims of current measurement.

Protocol (per participant, each with their own independent agent instance; two sessions 1–3 days apart). Session 1 opens with an agent-led onboarding interview (5–8 questions on communication style, clarification preference, domains of interest, tools, boundaries), stored as durable preferences and summarised back to the user; the agent then proposes 1–2 tasks based on what it learned. Session 2 skips re-onboarding: the agent greets from memory, the user continues the previous task and takes on one new related task, then a user-led exit interview has the user question the agent about what it remembers — and rate correctness, continuity ("same assistant, or a new one each time?"), and willingness to keep using it.

Targeted outputs. Counts of re-explanations and "I already told you" corrections in Session 2; whether the agent's interview answers match the user's recollection — a ground-truth check that parallels the IVG honesty gate (Section 3.6); a skill-provenance probe (Section 6.4), in which the user asks where the agent's claimed competence comes from and the answer is checked against the version-controlled skill library; and short quotable reactions contrasting continuity with fresh-session experience. Even N=2 with divergent participants (one expert, one non-expert) would materially strengthen the case study; running the protocol with non-expert participants is our planned next step, since the evidence to date comes from a single technical user (Sections 1.1, 6.1).

6.3 Pilot Results and Replication Round

Pilot (N=3 per condition). A pilot run has been completed; the derivation of its numbers is documented in the pilot analysis (`benchmark-results/pilot-three-run-analysis.md`). Once operational confounds were equalised, all three conditions completed the task — shifting differentiation from binary completion to consistency and quality. We report observed ranges only: at this sample size no statistical inference is possible, and the pilot results below are directional plausibility evidence rather than an evaluation.

Metric	C1 (Blank, n=3)	C2 (Configured, n=2†)	C3 (Evolved, n=2†)
Mean API calls	76 (range 40–115)	114 (range 109–119)	69 (range 68–70)
Mean input tokens	2,626,279	2,422,795	3,152,655
Mean output tokens	32,253	36,124	26,962
Tool errors (mean)	12.3	21.0	7.0
Production patterns	Basic	Async, UUID	Alembic, Docker, M3, zoneless
API call spread (observed)	2.9× (40–115)	1.1× (109–119)	**1.03× (68–70)**

† The run completed but its log was not captured before structured saving was implemented; token telemetry for that run is unavailable.

Key findings (pilot): 1. **Consistency was the most visible difference in the pilot runs.** C3's API call count stayed within ±1 (68–70) while C1 spanned 40–115 — a directional pattern consistent with skills providing a structural scaffold that reduces stochastic behaviour. We flag the caveat explicitly: with N=3 this is a hint, not a result — and, as the replication round below shows, it did not survive larger N. 2. **C3 produced production-grade patterns** (Alembic, Docker, Material M3) that were absent from all C1 and C2 runs — skills encode project-specific conventions that measurably improve output quality. 3. **Subagent delegation is stochastic**, appearing only when skills lack sufficient coverage. 4. **Cost is comparable across conditions**, all using the same model. The efficiency gain is in *user time*, not API spend.

Threats to validity: Small N; the pilot's cumulative container logs required session-ID deduplication, introducing classification uncertainty; the continuity test needs a fresh-session protocol; skill granularity (irrelevant skills inflate the count); model specificity (DeepSeek V4 Flash only); the configured condition's delegation accounting (subagent API calls are not captured in parent logs); missing log data (2 of 9 pilot trials, 2 of 15 replication trials).

Replication round (N=5 per condition, 13 of 15 logs captured). We repeated the benchmark with five runs per condition. First-run logs were lost for both the evolved and configured conditions; additionally, the configured condition's delegation made its parent-log API-call counts unreliable (subagent

calls are not captured in parent logs), so the API-call comparison here covers the blank and evolved conditions. Counting completed sessions only — one blank-condition session that hit the agent's iteration budget and a zero-call container-bootstrap session are excluded — API calls spanned 25–73 in the blank condition (sessions: 25, 40, 73, 37, 41; median 40) and 31–70 in the evolved condition (sessions: 31, 37, 70, 52, 31; median 37). For completeness, the configured condition's parent-log sessions across the captured logs ranged from 3 to 40 API calls (three budget-hit sessions excluded), but session-to-run attribution is uncertain — the first-run log is missing and subagent delegation fragments sessions — so these counts are not directly comparable; the raw logs are at `benchmark-results/day2-logs/`.

The ranges overlap: the pilot's API-call consistency signal did not replicate, and we retract it as a claimed differentiator. What replicated was output quality: the evolved condition again produced production-grade patterns (Alembic migrations, Docker Compose, Angular Material M3 theming, dark mode — including a learned aesthetic preference absent from its prompt) and the tightest output variance (41–49 files versus 26–72 for blank runs; file counts as recorded in the run summary `benchmark-results/day2-n5-complete.md`). We report this as directional plausibility evidence from a single user; no statistical inference is possible.

6.4 Skill Provenance and Calibrated Trust

When asked where it learned Angular, the evolved agent cited specific skill files — not pre-training data — concluding: *"My experience is our shared project work"* (verbatim; the exchange is short and reproducible). This procedural provenance — tracing *why* it knows rather than just *what* it knows — has implications for trust calibration. Rather than accepting opaque competence claims, users could evaluate the collaboration history behind claimed expertise. We treat this as a preliminary probe: it was observed once without a controlled protocol, and we flag it for validation under the evaluation agenda (Section 6.2).

6.5 Future Directions

Future work includes: CRDT-based cross-device sync; shared context pools for multi-user collaboration; skill ablation analysis (isolating session history from skills); the meta-agent pattern (self-improving boundaries and skills).

---

7. Conclusion — Answering the Research Question

We began by asking: *How can AI interaction be more powerful, efficient, and frictionless — especially for users who should not need to understand the machinery underneath?*

The answer is architectural persistence.

Over three months of iterative design, an architecture built for practical persistence independently converged on organisational patterns — memory hierarchies, impasse-driven escalation, procedural chunking, activation-weighted retrieval — that mirror decades of cognitive architecture research. These patterns emerged from concrete problems: stopping repeated questions, handling tasks that exceeded the agent's capability, compressing repetitive workflows, making recall feel natural. That they correspond to Soar and ACT-R is not coincidence — though we offer this as a falsifiable observation rather than a demonstrated result, since post-hoc pattern-matching is vulnerable to confirmation bias. If the correspondence holds, it suggests that these patterns are fundamental to any general cognitive system, and that architectures ignoring them will feel unnatural precisely because they deviate from cognitive expectations.

On the three dimensions of our research question:

- **Powerful:** the architecture compounds expertise through skill accumulation (the system gets measurably more capable over three months), extends reach through tiered delegation (the user never manages which model handles which task), and produces genuinely understanding interaction through cognitive convergence (the system feels like it "gets" the user because it is organised the way minds expect to be organised).
- **Efficient:** the user invests once and benefits across sessions. Zero re-explanation cycles across three months of daily use. Every correction, every preference established, every workflow refined in one session carries forward to the next. The system's API costs remain low (~$2–5/month), but the meaningful efficiency is the user's: time not spent re-establishing, re-explaining, re-configuring.
- **Frictionless:** one entry point, no profile management, no context switches. A replication round (N=5) found that output quality and variance, not API-call consistency, differentiated conditions; an earlier pilot hint of consistency did not survive larger N. The architecture enacts Alan Kay's principle: *simple things should be simple, complex things should be possible.* The user never sees the tiers, the memory systems, the delegation protocol. Simple interaction — just talk — is genuinely simple. Complex interaction is genuinely possible, handled through invisible internal escalation.

References
